

Name : Kanika Saini

Program: BTech CSE (B16)

SAP ID: 590034098

HDOC Day-9

Question 1

Write a menu-driven C program to input two numbers and swap them using a third variable and without using a third variable. Also display the values before and after swapping.

1. Problem Statement

Develop a menu-driven C program that allows the user to swap two numbers using two different methods:

- Swapping using a third (temporary) variable.
- Swapping without using a third variable.

The program should display the values before and after swapping based on the user's menu choice.

2. Input

First Number (a)

Second Number (b)

User Choice (1 or 2)

3. Algorithm

Step 1: Start the program.

Step 2: Display the menu:

- Swap using third variable
- Swap without third variable

Step 3: Read the user's choice.

Step 4: Input two numbers.

Step 5: If the choice is 1, perform swapping using a temporary variable.

Step 6: Else if the choice is 2, perform swapping without using a temporary variable.

Step 7: Display the values before and after swapping.

Step 8: If the choice is invalid, display an appropriate error message.

Step 9: Stop the program.

4. Pseudocode

Start

Display Menu

1. Swap using third variable
2. Swap without third variable

Read choice

Read first number

Read second number

Display values before swapping

If choice == 1

temp = a

a = b

b = temp

Else if choice == 2

a = a + b

b = a - b

a = a - b

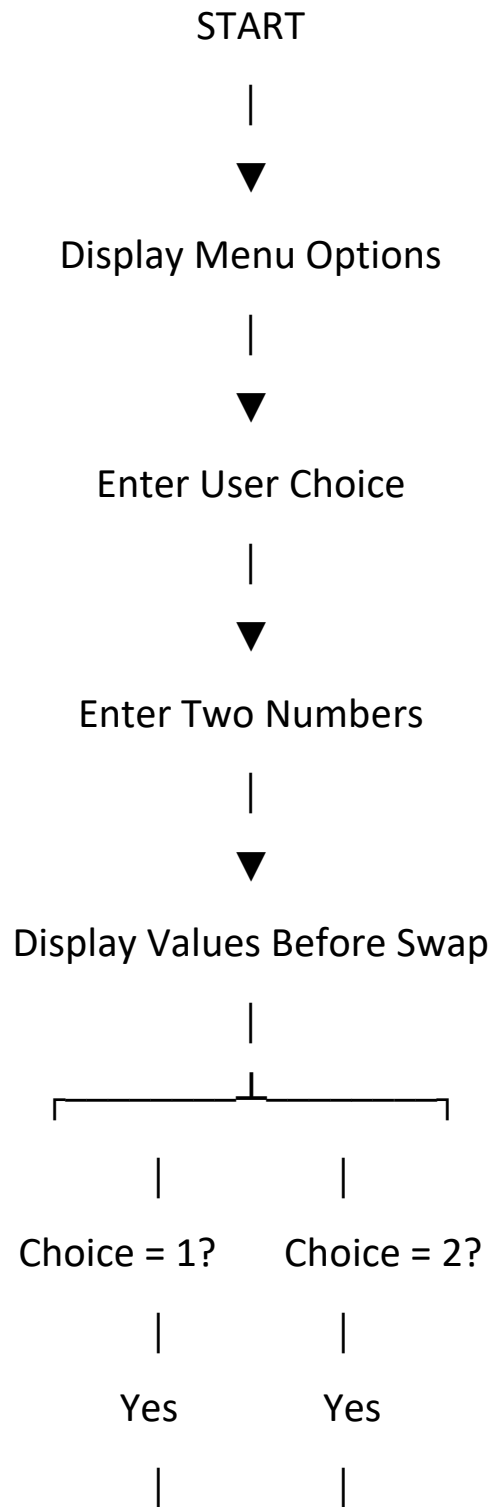
Else

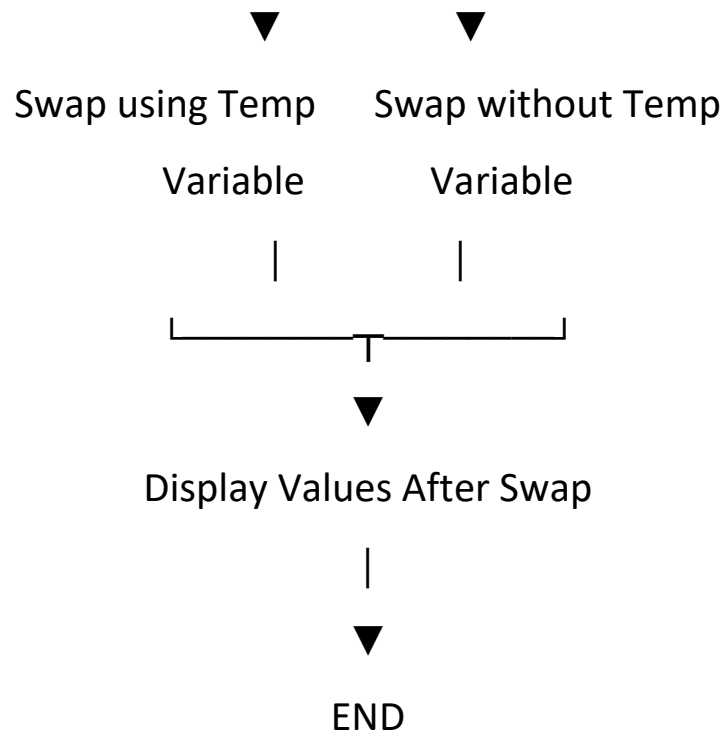
Display Invalid Choice

Display values after swapping

Stop

5. Flowchart





6. Test Case Table

Test case	Choice	Input (a,b)	Expected output	Actual output	Result
1.	1	10,20	20,10	Same	Pass
2.	2	15,25	25,15	Same	Pass
3.	1	5,5	5,5	Same	Pass
4.	2	-8,12	12,-8	Same	Pass
5.	3	10,20	Invalid choice	Same	Pass

7. C Program

```
#include <stdio.h>
```

```
int main()
```

```
{
    int a, b, temp, choice;
    printf("Menu\n");
    printf("1. Swap using third variable\n");
    printf("2. Swap without third variable\n");

    printf("Enter your choice: ");
    scanf("%d", &choice);

    printf("Enter first number: ");
    scanf("%d", &a);

    printf("Enter second number: ");
    scanf("%d", &b);

    printf("\nBefore Swapping\n");
    printf("a = %d\n", a);
    printf("b = %d\n", b);

    switch(choice)
    {
        case 1:
```

```
temp = a;
```

```
a = b;
```

```
b = temp;
```

```
break;
```

```
case 2:
```

```
a = a + b;
```

```
b = a - b;
```

```
a = a - b;
```

```
break;
```

```
default:
```

```
printf("Invalid Choice");
```

```
return 0;
```

```
}
```

```
printf("\nAfter Swapping\n");
```

```
printf("a = %d\n", a);
```

```
printf("b = %d\n", b);
```

```
return 0;
```

```
}
```

8. Compilation and execution

```
(kali㉿kali)-[~/Desktop]
$ cc swap.c
```

```
(kali㉿kali)-[~/Desktop]
$ ./a.out
```

Menu

1. Swap using third variable
2. Swap without third variable
Enter your choice: 1
Enter first number: 10
Enter second number: 20

Before Swapping

a = 10
b = 20

After Swapping

a = 20
b = 10

```
(kali㉿kali)-[~/Desktop]
$ ./a.out
```

Menu

1. Swap using third variable
2. Swap without third variable
Enter your choice: 2
Enter first number: 15
Enter second number: 25

Before Swapping

a = 15
b = 25

After Swapping

a = 25
b = 15

```
(kali㉿kali)-[~/Desktop]
$ ./a.out
```

Menu

1. Swap using third variable
2. Swap without third variable
Enter your choice: 1
Enter first number: 5
Enter second number: 5

Before Swapping

a = 5
b = 5

After Swapping

a = 5
b = 5

```
(kali㉿kali)-[~/Desktop]
$ ./a.out
```

Menu

1. Swap using third variable
2. Swap without third variable
Enter your choice: 2
Enter first number: -8
Enter second number: 12

Before Swapping

a = -8
b = 12

After Swapping

a = 12
b = -8

```
(kali㉿kali)-[~/Desktop]
$ ./a.out
```

Menu

1. Swap using third variable
2. Swap without third variable
Enter your choice: 3
Enter first number: 10
Enter second number: 20

Before Swapping

a = 10
b = 20

Invalid Choice

9. Evaluation of the Code Against the Test Cases

Test Case 1: The program successfully swapped the numbers using a third variable and matched the expected output. Result: Pass

Test Case 2: The program correctly swapped the numbers without using a third variable. Result: Pass

Test Case 3: When both input values were the same, the program retained the values correctly after swapping. Result: Pass

Test Case 4: The program correctly handled negative numbers and produced the expected output. Result: Pass

Test Case 5: The program detected an invalid menu choice and displayed an appropriate error message. Result: Pass

Question 2

Write a menu-driven C program to find the sum of the first n natural numbers and the sum of squares of the first n natural numbers.

1. Problem Statement

Develop a menu-driven C program that allows the user to calculate:

1. The sum of the first n natural numbers.
2. The sum of the squares of the first n natural numbers.

The program should display the result based on the user's menu selection.

2. Input :

- User Choice (1 or 2)
- Value of n

3. Algorithm

Step 1: Start the program.

Step 2: Display the menu:

- Sum of first n natural numbers
- Sum of squares of first n natural numbers

Step 3: Read the user's choice.

Step 4: Input the value of n.

Step 5: If the choice is 1, use a loop to calculate the sum of the first n natural numbers.

Step 6: If the choice is 2, use a loop to calculate the sum of the squares of the first n natural numbers.

Step 7: Display the calculated result.

Step 8: If the choice is invalid, display an error message.

Step 9: Stop the program.

4. Pseudocode

Start

Display Menu

1. Sum of first n natural numbers
2. Sum of squares of first n natural numbers

Read choice

Read n

sum = 0

If choice == 1

Repeat from i = 1 to n

sum = sum + i

Display sum

Else if choice == 2

Repeat from i = 1 to n

sum = sum + i*i

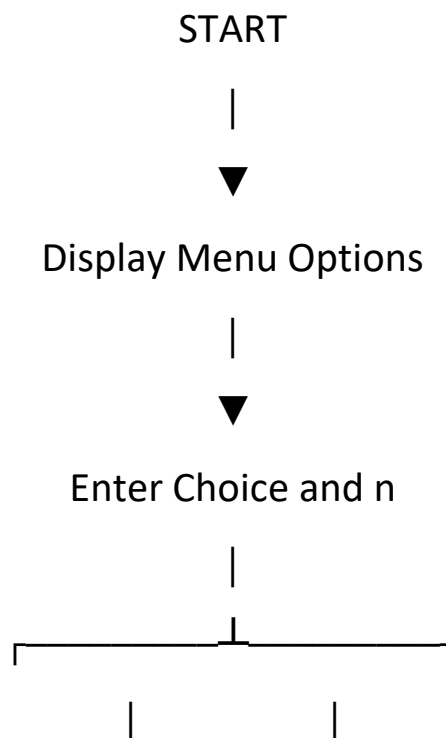
Display sum

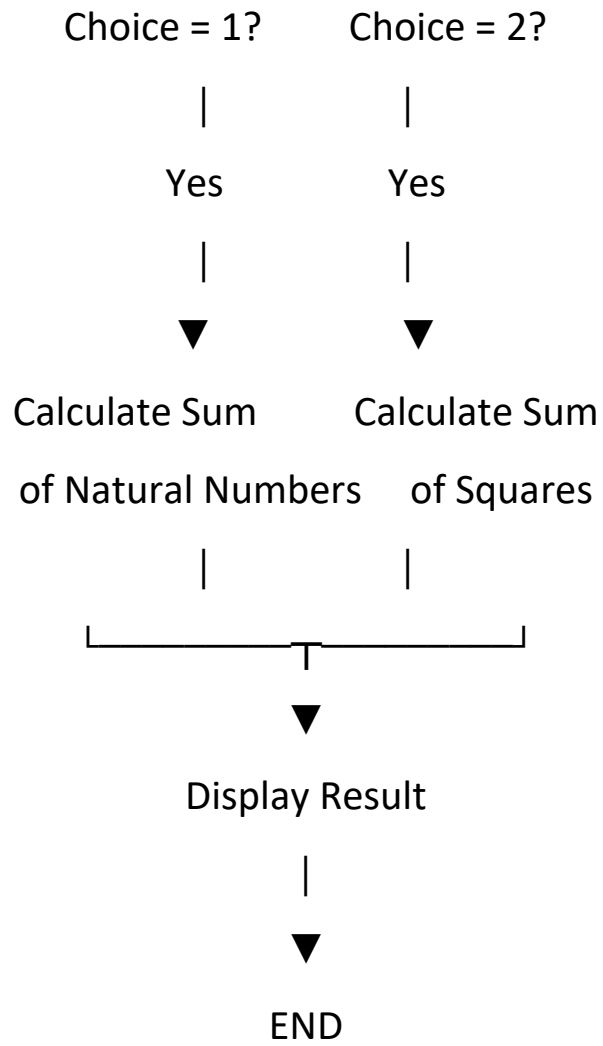
Else

Display Invalid Choice

Stop

5. Flowchart





6. Test Case Table

Test case	Choice	Input (n)	Expected output	Actual output	Result
1.	1	5	15	Same	Pass
2.	2	5	55	Same	Pass
3.	1	10	55	Same	Pass
4.	2	3	14	Same	Pass
5.	3	5	Invalid choice	Same	Pass

7. C Program

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int choice, n, i, sum = 0;
```

```
    printf("Menu\n");
```

```
    printf("1. Sum of first n natural numbers\n");
```

```
    printf("2. Sum of squares of first n natural numbers\n");
```

```
    printf("Enter your choice: ");
```

```
    scanf("%d", &choice);
```

```
printf("Enter the value of n: ");
```

```
scanf("%d", &n);
```

```
switch(choice)
```

```
{
```

```
    case 1:
```

```
        for(i = 1; i <= n; i++)
```

```
            sum += i;
```

```
        printf("Sum = %d", sum);
```

```
        break;
```

```
    case 2:
```

```
        for(i = 1; i <= n; i++)
```

```
            sum += i * i;
```

```
        printf("Sum of Squares = %d", sum);
```

```
        break;
```

```
    default:
```

```
        printf("Invalid Choice");
```

```
}
```

```
    return 0;  
}
```

8. Compilation and execution

```
(kali㉿kali)-[~/Desktop]  
$ cc sum.c  
  
(kali㉿kali)-[~/Desktop]  
$ ./a.out  
Menu  
1. Sum of first n natural numbers  
2. Sum of squares of first n natural numbers  
Enter your choice: 1  
Enter the value of n: 5  
Sum = 15
```

```
(kali㉿kali)-[~/Desktop]  
$ ./a.out  
Menu  
1. Sum of first n natural numbers  
2. Sum of squares of first n natural numbers  
Enter your choice: 2  
Enter the value of n: 5  
Sum of Squares = 55
```

```
(kali㉿kali)-[~/Desktop]  
$ ./a.out  
Menu  
1. Sum of first n natural numbers  
2. Sum of squares of first n natural numbers  
Enter your choice: 1  
Enter the value of n: 10  
Sum = 55
```



```
(kali㉿kali)-[~/Desktop]
$ ./a.out
Menu
1. Sum of first n natural numbers
2. Sum of squares of first n natural numbers
Enter your choice: 2
Enter the value of n: 3
Sum of Squares = 14

(kali㉿kali)-[~/Desktop]
$ ./a.out
Menu
1. Sum of first n natural numbers
2. Sum of squares of first n natural numbers
Enter your choice: 3
Enter the value of n: 5
Invalid Choice
```

9. Evaluation of the Code Against the Test Cases

Test Case 1: The program correctly calculated the sum of the first 5 natural numbers. Result: Pass

Test Case 2: The program correctly calculated the sum of the squares of the first 5 natural numbers. Result: Pass

Test Case 3: The program produced the expected sum for the first 10 natural numbers. Result: Pass

Test Case 4: The program correctly calculated the sum of the squares for the first 3 natural numbers. Result: Pass

Test Case 5: The program displayed an appropriate error message for an invalid menu choice. Result: Pass

Question 3

Write a menu-driven C program to calculate Simple Interest, Compound Amount, Compound Interest, and Total Payable Amount for the given Principal, Rate, and Time.

1. Problem Statement

Develop a menu-driven C program that allows the user to calculate:

1. Simple Interest (SI)
2. Compound Amount (CA)
3. Compound Interest (CI)
4. Total Payable Amount

The program should take the Principal (P), Rate of Interest (R), and Time (T) as inputs and perform the selected calculation.

2. Input

- Principal (P)
- Rate of Interest (R)
- Time (T)
- User Choice (1–4)

3. Algorithm

Step 1: Start the program.

Step 2: Display the menu:

- Simple Interest
- Compound Amount
- Compound Interest
- Total Payable Amount

Step 3: Read the user's choice.

Step 4: Input Principal (P), Rate (R), and Time (T).

Step 5: According to the selected option:

- Calculate Simple Interest using $SI = (P \times R \times T) / 100$.
- Calculate Compound Amount using $CA = P \times (1 + R/100)^T$.
- Calculate Compound Interest using $CI = CA - P$.
- Calculate Total Payable Amount using $P + SI$.

Step 6: Display the calculated result.

Step 7: If the entered choice is invalid, display an error message.

Step 8: Stop the program.

4. Pseudocode

Start

Display Menu

1. Simple Interest
2. Compound Amount
3. Compound Interest
4. Total Payable Amount

Read choice

Read Principal, Rate and Time

If choice == 1

$$SI = (P \times R \times T) / 100$$

Display SI

Else if choice == 2

$$CA = P \times (1 + R/100)^T$$

Display CA

Else if choice == 3

$$CA = P \times (1 + R/100)^T$$

$$CI = CA - P$$

Display CI

Else if choice == 4

$$SI = (P \times R \times T) / 100$$

$$\text{Total} = P + SI$$

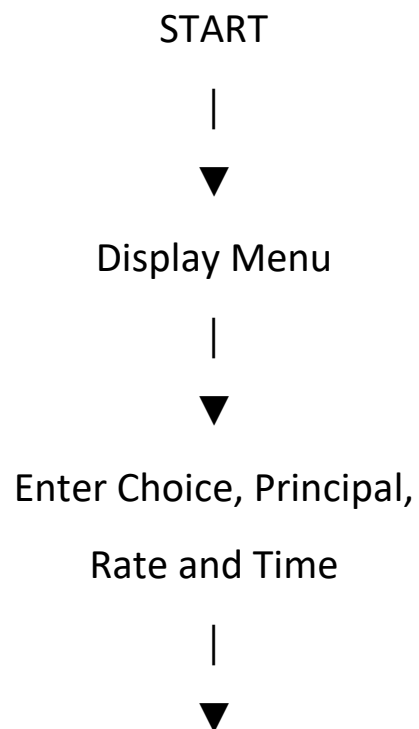
Display Total

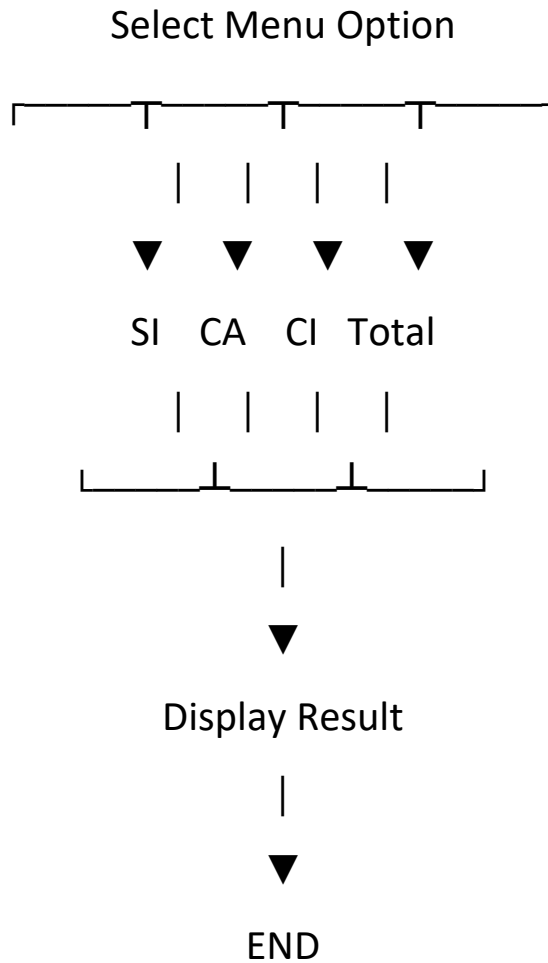
Else

Display Invalid Choice

Stop

5. Flowchart





6. Test Case Table

Test case	Choice	Input (P,R,T)	Expected output	Actual output	Result
1.	1	1000,10,2	SI=200	Same	Pass
2.	2	1000,10,2	CA=1210	Same	Pass

3.	1	1000,10,2	CI=210	Same	Pass
4.	2	1000,10,2	TOTAL=1200	Same	Pass
5.	3	1000,10,2	Invalid choice	Same	Pass

7. C Program

```
#include <stdio.h>
```

```
#include <math.h>
```

```
int main()
```

```
{
```

```
    int choice;
```

```
    float P, R, T, SI, CA, CI, Total;
```

```
    printf("Menu\n");
```

```
    printf("1. Simple Interest\n");
```

```
    printf("2. Compound Amount\n");
```

```
    printf("3. Compound Interest\n");
```

```
    printf("4. Total Payable Amount\n");
```

```
    printf("Enter your choice: ");
```

```
    scanf("%d", &choice);
```

```
printf("Enter Principal, Rate and Time: ");
```

```
scanf("%f%f%f", &P, &R, &T);
```

```
switch(choice)
```

```
{
```

```
case 1:
```

```
    SI = (P * R * T) / 100;
```

```
    printf("Simple Interest = %.2f", SI);
```

```
    break;
```

```
case 2:
```

```
    CA = P * pow((1 + R / 100), T);
```

```
    printf("Compound Amount = %.2f", CA);
```

```
    break;
```

```
case 3:
```

```
    CA = P * pow((1 + R / 100), T);
```

```
    CI = CA - P;
```

```
    printf("Compound Interest = %.2f", CI);
```

```
    break;
```


case 4:

$SI = (P * R * T) / 100;$

Total = P + SI;

printf("Total Payable Amount = %.2f", Total);

break;

default:

printf("Invalid Choice");

}

return 0;

}

8. Compilation and execution

```
(kali㉿kali)-[~/Desktop]
$ cc interest.c -lm

(kali㉿kali)-[~/Desktop]
$ ./a.out
Menu
1. Simple Interest
2. Compound Amount
3. Compound Interest
4. Total Payable Amount
Enter your choice: 1
Enter Principal, Rate and Time: 1000 10 2
Simple Interest = 200.00
```

```
(kali㉿kali)-[~/Desktop]
$ ./a.out
Menu
1. Simple Interest
2. Compound Amount
3. Compound Interest
4. Total Payable Amount
Enter your choice: 2
Enter Principal, Rate and Time: 1000 10 2
Compound Amount = 1210.00
```

```
(kali㉿kali)-[~/Desktop]
$ ./a.out
Menu
1. Simple Interest
2. Compound Amount
3. Compound Interest
4. Total Payable Amount
Enter your choice: 3
Enter Principal, Rate and Time: 1000 10 2
Compound Interest = 210.00
```

```
(kali㉿kali)-[~/Desktop]
$ ./a.out
Menu
1. Simple Interest
2. Compound Amount
3. Compound Interest
4. Total Payable Amount
Enter your choice: 4
Enter Principal, Rate and Time: 1000 10 2
Total Payable Amount = 1200.00

(kali㉿kali)-[~/Desktop]
$ ./a.out
Menu
1. Simple Interest
2. Compound Amount
3. Compound Interest
4. Total Payable Amount
Enter your choice: 5
Enter Principal, Rate and Time: 1000 10 2
Invalid Choice
```

9. Evaluation of the Code Against the Test Cases

Test Case 1: The program correctly calculated the Simple Interest for the given principal, rate, and time. Result: Pass

Test Case 2: The program accurately computed the Compound Amount using the compound interest formula. Result: Pass

Test Case 3: The program correctly determined the Compound Interest by subtracting the principal from the compound amount. Result: Pass

Test Case 4: The program correctly calculated the Total Payable Amount by adding the principal and simple interest. Result: Pass

Test Case 5: The program displayed an appropriate error message for an invalid menu choice. Result: Pass